

VELORA — Module Subscription (Part 2)

**Service · Controller · Resources · Routes · Middleware
· Seeder · Commands**

8. SubscriptionService

app/Modules/Subscription/Services/SubscriptionService.php

```
<?php
```

```
namespace App\Modules\Subscription\Services;
```

```
use App\Modules\Subscription\Enums\BillingCycle;
```

```
use App\Modules\Subscription\Enums\BillingMode;
```

```
use App\Modules\Subscription\Enums\SubscriptionStatus;
```

```
use App\Modules\Subscription\Events\SubscriptionActivated;
```

```
use App\Modules\Subscription\Events\SubscriptionExpired;
```

```
use App\Modules\Subscription\Events\TrialStarted;
```

```
use App\Modules\Subscription\Models\Plan;
```

```
use App\Modules\Subscription\Models\Subscription;
```

```
use App\Modules\Subscription\Models\SubscriptionInvoice;
```

```
use App\Modules\Subscription\Repositories\Contracts\PlanRepositoryInterface;
```

```
use App\Modules\Subscription\Repositories\Contracts\SubscriptionRepositoryInterface;
```

```
use App\Shared\Exceptions\VeloraException;
```

```
use App\Shared\Services\BaseService;
```

```
use Illuminate\Database\Eloquent\Collection;
```

```
use Illuminate\Support\Facades\DB;
```

```
use Illuminate\Support\Str;
```

```
class SubscriptionService extends BaseService
```

```
{
```

```
    public function __construct(
```

```
        private readonly PlanRepositoryInterface $planRepo,
```

```
        private readonly SubscriptionRepositoryInterface $subRepo,
```

```
    ) {}
```

```
    /* -----  
    | GET PLANS  
    ----- */
```

```
    public function getActivePlans(): Collection
```

```
    {
```

```
        return $this->planRepo->allActive();
```

```
    }
```

```
    public function findPlanOrFail(string $id): Plan
```

```

{
    $plan = $this->planRepo->findById($id);

    if (!$plan) {
        throw VeloraException::notFound('Plan');
    }

    return $plan;
}

/* _____
| CURRENT SUBSCRIPTION
_____ */

public function getCurrentSubscription(string $businessId): ?Subscription
{
    return $this->subRepo->findByBusiness($businessId);
}

/* _____
| START TRIAL (dipanggil saat register)
_____ */

public function startTrial(string $businessId, string $planSlug = 'starter'): Subscription
{
    $plan = $this->planRepo->findBySlug($planSlug);

    if (!$plan) {
        throw VeloraException::notFound('Plan');
    }

    $subscription = $this->subRepo->create([
        'business_id' => $businessId,
        'plan_id'      => $plan->id,
        'status'       => SubscriptionStatus::Trial->value,
        'billing_cycle' => BillingCycle::Monthly->value,
        'billing_mode' => BillingMode::Manual->value,
        'trial_ends_at' => now()->addDays($plan->trial_days),
        'price_paid'   => 0,
    ]);

    event(new TrialStarted($subscription));
}

```

```

        return $subscription;
    }

    /* -----
    | SUBSCRIBE (buat invoice + tunggu bayar)
    ----- */

    public function subscribe(
        string      $businessId,
        string      $planId,
        BillingCycle $cycle,
        BillingMode  $mode,
    ): array {
        return DB::transaction(function () use ($businessId, $planId, $cycle, $mode) {

            $plan = $this->findPlanOrFail($planId);
            $price = $cycle === BillingCycle::Yearly
                ? $plan->price_yearly
                : $plan->price_monthly;

            // Buat / update subscription
            $existing = $this->subRepo->findByBusiness($businessId);

            if ($existing && $existing->isActive()) {
                // Upgrade/downgrade – extend existing
                $subscription = $this->subRepo->update($existing->id, [
                    'plan_id'      => $plan->id,
                    'billing_cycle' => $cycle->value,
                    'billing_mode'  => $mode->value,
                    'status'        => SubscriptionStatus::Active->value,
                ]);
            } else {
                $subscription = $this->subRepo->create([
                    'business_id'  => $businessId,
                    'plan_id'      => $plan->id,
                    'status'        => SubscriptionStatus::Active->value,
                    'billing_cycle' => $cycle->value,
                    'billing_mode'  => $mode->value,
                    'starts_at'     => now(),
                    'ends_at'       => now()->addMonths($cycle->months()),
                    'grace_ends_at' => now()->addMonths($cycle->months())->addDays(3),
                    'price_paid'    => $price,
                ]);
            }
        });
    }

```

```

    }

    // Buat invoice
    $invoice = $this->createInvoice($subscription, $plan, $price, $cycle);

    return compact('subscription', 'invoice');
});
}

/* _____
|  ACTIVATE (setelah bayar)
|_____ */

public function activate(string $subscriptionId, ?string $transactionId = null): Subscription
{
    $subscription = $this->subRepo->findById($subscriptionId);

    if (!$subscription) {
        throw VeloraException::notFound('Subscription');
    }

    $cycle = $subscription->billing_cycle;

    $updated = $this->subRepo->update($subscription->id, [
        'status'           => SubscriptionStatus::Active->value,
        'starts_at'         => now(),
        'ends_at'           => now()->addMonths($cycle->months()),
        'grace_ends_at'     => now()->addMonths($cycle->months())->addDays(3),
        'midtrans_transaction_id' => $transactionId,
    ]);

    event(new SubscriptionActivated($updated));

    return $updated;
}

/* _____
|  CANCEL
|_____ */

public function cancel(string $businessId): Subscription
{
    $sub = $this->subRepo->findByBusiness($businessId);

```

```

    if (!$sub || !$sub->isActive()) {
        throw new VeloraException('Tidak ada subscription aktif yang bisa dibatalkan.', 422);
    }

    return $this->subRepo->update($sub->id, [
        'status'          => SubscriptionStatus::Cancelled->value,
        'cancelled_at' => now(),
    ]);
}

/* -----
|  PROCESS EXPIRED (via Artisan command)
| ----- */

public function processExpiredSubscriptions(): void
{
    // Active → Grace
    foreach ($this->subRepo->findExpiredActive() as $sub) {
        $this->subRepo->update($sub->id, ['status' => SubscriptionStatus::Grace->value]);
    }

    // Grace → Expired
    foreach ($this->subRepo->findExpiredGrace() as $sub) {
        $this->subRepo->update($sub->id, ['status' => SubscriptionStatus::Expired->value]);
        event(new SubscriptionExpired($sub));
    }
}

/* -----
|  CHECK FEATURE LIMIT
| ----- */

public function checkLimit(string $businessId, string $limitField, int $currentCount): bool
{
    $sub = $this->subRepo->findByBusiness($businessId);

    if (!$sub || !$sub->isActive()) {
        return false;
    }

    return $sub->isWithinLimit($limitField, $currentCount);
}

```

```

public function canUseFeature(string $businessId, string $feature): bool
{
    $sub = $this->subRepo->findByBusiness($businessId);
    return $sub?->canUseFeature($feature) ?? false;
}

/* -----
| PRIVATE HELPERS
----- */

private function createInvoice(
    Subscription $sub,
    Plan          $plan,
    int           $price,
    BillingCycle  $cycle,
): SubscriptionInvoice {
    return SubscriptionInvoice::create([
        'subscription_id' => $sub->id,
        'business_id'     => $sub->business_id,
        'invoice_number'  => 'INV/' . date('Ymd') . '/' . strtoupper(Str::random(6)),
        'status'          => 'unpaid',
        'amount'          => $price,
        'currency'         => 'IDR',
        'billing_cycle'    => $cycle->value,
        'due_at'           => now()->addDays(3),
    ]);
}
}

```

9. Resources

app/Modules/Subscription/Resources/PlanResource.php

```
<?php
```

```
namespace App\Modules\Subscription\Resources;
```

```
use App\Shared\Helpers\MoneyHelper;
```

```
use Illuminate\Http\Request;
```

```
use Illuminate\Http\Resources\Json\JsonResource;
```

```
class PlanResource extends JsonResource
```

```
{
    public function toArray(Request $request): array
    {
        return [
            'id'                => $this->id,
            'name'              => $this->name,
            'slug'              => $this->slug,
            'description'       => $this->description,
            'pricing' => [
                'monthly'        => $this->price_monthly,
                'monthly_fmt'    => MoneyHelper::format($this->price_monthly),
                'yearly'         => $this->price_yearly,
                'yearly_fmt'     => MoneyHelper::format($this->price_yearly),
                'trial_days'     => $this->trial_days,
            ],
            'limits' => [
                'max_branches'   => $this->max_branches,
                'max_products'   => $this->max_products,
                'max_employees'  => $this->max_employees,
                'max_customers'  => $this->max_customers,
                'max_transactions_per_month' => $this->max_transactions_per_month,
            ],
            'features' => [
                'ai'             => $this->has_ai,
                'multi_currency' => $this->has_multi_currency,
                'report_export'  => $this->has_report_export,
                'api_access'     => $this->has_api_access,
                'barcode_scanner' => $this->has_barcode_scanner,
                'purchase_order' => $this->has_purchase_order,
            ],
        ];
    }
}
```



```

        'crm'                => $this->has_crm,
    ],
];
}
}

```

app/Modules/Subscription/Resources/SubscriptionResource.php

```
<?php
```

```
namespace App\Modules\Subscription\Resources;
```

```
use Illuminate\Http\Request;
```

```
use Illuminate\Http\Resources\Json\JsonResource;
```

```
class SubscriptionResource extends JsonResource
```

```

{
    public function toArray(Request $request): array
    {
        return [
            'id'                => $this->id,
            'status'            => $this->status,
            'status_label'      => $this->status->label(),
            'is_active'         => $this->isActive(),
            'is_in_trial'       => $this->isInTrial(),
            'days_remaining'   => max(0, $this->daysUntilExpiry()),
            'billing_cycle'     => $this->billing_cycle,
            'billing_mode'      => $this->billing_mode,
            'trial_ends_at'     => $this->trial_ends_at?->toIso8601String(),
            'starts_at'         => $this->starts_at?->toIso8601String(),
            'ends_at'           => $this->ends_at?->toIso8601String(),
            'grace_ends_at'     => $this->grace_ends_at?->toIso8601String(),
            'plan'              => new PlanResource($this->whenLoaded('plan')),
        ];
    }
}

```

10. Requests

app/Modules/Subscription/Requests/SubscribeRequest.php

```
<?php

namespace App\Modules\Subscription\Requests;

use App\Modules\Subscription\Enums\BillingCycle;
use App\Modules\Subscription\Enums\BillingMode;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rules\Enum;

class SubscribeRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'plan_id' => ['required', 'uuid', 'exists:plans,id'],
            'billing_cycle' => ['required', new Enum(BillingCycle::class)],
            'billing_mode' => ['required', new Enum(BillingMode::class)],
        ];
    }
}
```

11. Controller

app/Modules/Subscription/Controllers/SubscriptionController.php

```
<?php
```

```
namespace App\Modules\Subscription\Controllers;
```

```
use App\Modules\Subscription\Enums\BillingCycle;
```

```
use App\Modules\Subscription\Enums\BillingMode;
```

```
use App\Modules\Subscription\Requests\SubscribeRequest;
```

```
use App\Modules\Subscription\Resources\PlanResource;
```

```
use App\Modules\Subscription\Resources\SubscriptionResource;
```

```
use App\Modules\Subscription\Services\SubscriptionService;
```

```
use App\Shared\Controllers\BaseController;
```

```
use Illuminate\Http\JsonResponse;
```

```
use Illuminate\Http\Request;
```

```
class SubscriptionController extends BaseController
```

```
{
```

```
    public function __construct(private readonly SubscriptionService $service) {}
```

```
    // GET /api/v1/plans
```

```
    public function plans(): JsonResponse
```

```
    {
```

```
        $plans = $this->service->getActivePlans();
```

```
        return $this->success(PlanResource::collection($plans), 'Available plans.');
```

```
    }
```

```
    // GET /api/v1/subscription
```

```
    public function current(Request $request): JsonResponse
```

```
    {
```

```
        $sub = $this->service->getCurrentSubscription($request->user()->business_id);
```

```
        if (!$sub) {
```

```
            return $this->error('No active subscription found.', 404, 'NO_SUBSCRIPTION');
```

```
        }
```

```
        return $this->success(new SubscriptionResource($sub));
```

```
    }
```

```
    // POST /api/v1/subscription/subscribe
```

```

public function subscribe(SubscribeRequest $request): JsonResponse
{
    $data = $request->validated();
    $result = $this->service->subscribe(
        $request->user()->business_id,
        $data['plan_id'],
        BillingCycle::from($data['billing_cycle']),
        BillingMode::from($data['billing_mode']),
    );

    return $this->created([
        'subscription' => new SubscriptionResource($result['subscription']),
        'invoice'      => [
            'id'          => $result['invoice']->id,
            'invoice_number' => $result['invoice']->invoice_number,
            'amount'       => $result['invoice']->amount,
            'due_at'       => $result['invoice']->due_at->toIso8601String(),
        ],
    ], 'Subscription created. Please complete payment.');
```



```

// POST /api/v1/subscription/cancel
public function cancel(Request $request): JsonResponse
{
    $sub = $this->service->cancel($request->user()->business_id);
    return $this->success(new SubscriptionResource($sub), 'Subscription cancelled.');
```



```

}
```

12. Routes

app/Modules/Subscription/Routes/api.php

```
<?php
```

```
use App\Modules\Subscription\Controllers\SubscriptionController;
```

```
use Illuminate\Support\Facades\Route;
```

```
// Public – siapapun bisa lihat plans
```

```
Route::get('/plans', [SubscriptionController::class, 'plans'])->name('plans.index');
```

```
// Protected
```

```
Route::middleware(['auth:sanctum', 'tenant.scope'])->prefix('subscription')->name('subscription.');
```

```
Route::get('/', [SubscriptionController::class, 'current'])->name('current');
```

```
Route::post('/subscribe', [SubscriptionController::class, 'subscribe'])->name('subscribe');
```

```
Route::post('/cancel', [SubscriptionController::class, 'cancel'])->name('cancel');
```

```
});
```

13. Middleware — EnsureActiveSubscription (Final)

app/Shared/Middleware/EnsureActiveSubscription.php

```
<?php
```

```
namespace App\Shared\Middleware;
```

```
use App\Shared\Helpers\ApiResponse;
```

```
use Closure;
```

```
use Illuminate\Http\Request;
```

```
use Illuminate\Support\Facades\Cache;
```

```
class EnsureActiveSubscription
```

```
{
```

```
    public function handle(Request $request, Closure $next)
```

```
    {
```

```
        $user = $request->user();
```

```
        if (!$user->business_id) {
```

```
            return ApiResponse::forbidden('No business context.', 'NO_TENANT');
```

```
        }
```

```
        $cacheKey = "subscription:active:{$user->business_id}";
```

```
        $isActive = Cache::remember($cacheKey, now()->addMinutes(5), function () use ($user) {
```

```
            $business = \App\Modules\Tenant\Models\Business::with('subscription.plan')
```

```
                ->find($user->business_id);
```

```
            return $business->hasActiveSubscription() ?? false;
```

```
        });
```

```
        if (!$isActive) {
```

```
            return ApiResponse::forbidden(
```

```
                'Subscription Anda telah kadaluarsa. Silakan perpanjang untuk melanjutkan.',
```

```
                'SUBSCRIPTION_EXPIRED'
```

```
            );
```

```
        }
```

```
        return $next($request);
```

```
    }
```

```
}
```

app/Shared/Middleware/CheckFeatureLimit.php

```
<?php
```

```
namespace App\Shared\Middleware;
```

```
use App\Shared\Helpers\ApiResponse;
```

```
use Closure;
```

```
use Illuminate\Http\Request;
```

```
class CheckFeatureLimit
```

```
{
```

```
    /**
```

```
     * Penggunaan: middleware('feature.limit:has_ai')
```

```
     * Cek apakah business punya feature tertentu dari plan-nya
```

```
     */
```

```
    public function handle(Request $request, Closure $next, string $feature)
```

```
    {
```

```
        $user      = $request->user();
```

```
        $business  = app('current.business');
```

```
        $plan      = $business->currentPlan();
```

```
        if (!$plan || !($plan->{$feature} ?? false)) {
```

```
            return ApiResponse::forbidden(
```

```
                "Fitur ini tidak tersedia di paket Anda. Upgrade untuk menggunakan fitur ini.",
```

```
                'FEATURE_NOT_AVAILABLE'
```

```
            );
```

```
        }
```

```
        return $next($request);
```

```
    }
```

```
}
```

14. Artisan Command — Check Expired Subscriptions

app/Console/Commands/CheckExpiredSubscriptions.php

```
<?php

namespace App\Console\Commands;

use App\Modules\Subscription\Services\SubscriptionService;
use Illuminate\Console\Command;

class CheckExpiredSubscriptions extends Command
{
    protected $signature    = 'velora:check-subscriptions';
    protected $description = 'Process expired subscriptions (Active→Grace, Grace→Expired)';

    public function handle(SubscriptionService $service): int
    {
        $this->info('[VELORA] Checking expired subscriptions...');

        $service->processExpiredSubscriptions();

        $this->info('[VELORA] Done.');
```

return self::SUCCESS;

```
    }
}
```

Schedule di routes/console.php

```
use Illuminate\Support\Facades\Schedule;

Schedule::command('velora:check-subscriptions')->daily()->at('00:05');
```


15. Seeder — Plans

database/seeder/SubscriptionPlanSeeder.php

```
<?php

namespace Database\Seeders;

use App\Modules\Subscription\Models\Plan;
use Illuminate\Database\Seeder;

class SubscriptionPlanSeeder extends Seeder
{
    public function run(): void
    {
        $plans = [
            [
                'name' => 'Starter',
                'slug' => 'starter',
                'description' => 'Cocok untuk usaha baru yang baru memulai.',
                'price_monthly' => 99_000,
                'price_yearly' => 990_000,
                'trial_days' => 14,
                'max_branches' => 1,
                'max_products' => 100,
                'max_employees' => 5,
                'max_customers' => 500,
                'max_transactions_per_month' => 500,
                'has_ai' => false,
                'has_multi_currency' => false,
                'has_report_export' => false,
                'has_api_access' => false,
                'has_barcode_scanner' => true,
                'has_purchase_order' => false,
                'has_crm' => false,
                'sort_order' => 1,
                'is_active' => true,
            ],
            [
                'name' => 'Growth',
                'slug' => 'growth',
                'description' => 'Untuk usaha yang sedang berkembang.',
```

```

        'price_monthly'           => 299_000,
        'price_yearly'           => 2_990_000,
        'trial_days'             => 14,
        'max_branches'           => 3,
        'max_products'           => 1000,
        'max_employees'          => 20,
        'max_customers'          => 5000,
        'max_transactions_per_month' => 5000,
        'has_ai'                  => false,
        'has_multi_currency'      => true,
        'has_report_export'       => true,
        'has_api_access'          => false,
        'has_barcode_scanner'     => true,
        'has_purchase_order'      => true,
        'has_crm'                 => true,
        'sort_order'              => 2,
        'is_active'               => true,
    ],
    [
        'name'                     => 'Enterprise',
        'slug'                     => 'enterprise',
        'description'               => 'Untuk multi-outlet dan bisnis skala besar.',
        'price_monthly'           => 799_000,
        'price_yearly'           => 7_990_000,
        'trial_days'             => 14,
        'max_branches'           => -1,
        'max_products'           => -1,
        'max_employees'          => -1,
        'max_customers'          => -1,
        'max_transactions_per_month' => -1,
        'has_ai'                  => true,
        'has_multi_currency'      => true,
        'has_report_export'       => true,
        'has_api_access'          => true,
        'has_barcode_scanner'     => true,
        'has_purchase_order'      => true,
        'has_crm'                 => true,
        'sort_order'              => 3,
        'is_active'               => true,
    ],
];

foreach ($plans as $plan) {

```

```

        Plan::updateOrCreate(['slug' => $plan['slug']], $plan);
    }

    $this->command->info('Plans seeded: Starter, Growth, Enterprise');
}
}

# Run seeder
php artisan db:seed --class=SubscriptionPlanSeeder

```

16. Update Auth — Start Trial saat Register

Update AuthService::register() — tambahkan trial start

```

// Tambahkan setelah event(new UserRegistered()) di AuthService::register()

app(\App\Modules\Subscription\Services\SubscriptionService::class)
    ->startTrial($business->id, 'starter');

// Atau lebih baik via Listener di EventServiceProvider:
// UserRegistered → StartTrialSubscription::class

```

app/Modules/Subscription/Listeners/StartTrialSubscription.php

```
<?php

namespace App\Modules\Subscription\Listeners;

use App\Modules\Auth\Events\UserRegistered;
use App\Modules\Subscription\Services\SubscriptionService;
use Illuminate\Contracts\Queue\ShouldQueue;

class StartTrialSubscription implements ShouldQueue
{
    public string $queue = 'default';

    public function __construct(private readonly SubscriptionService $service) {}

    public function handle(UserRegistered $event): void
    {
        $this->service->startTrial($event->business->id, 'starter');
    }
}
```

Daftarkan di EventServiceProvider :

```
UserRegistered::class => [
    \App\Modules\Auth\Listeners\SendWelcomeEmail::class,
    \App\Modules\Subscription\Listeners\StartTrialSubscription::class,
],
```

Checklist Module Subscription

- ✅ Migration: plans , subscriptions , subscription_invoices
- ✅ Enums: SubscriptionStatus , BillingCycle , BillingMode
- ✅ Model: Plan (feature flags, limit helpers, active scope)
- ✅ Model: Subscription (status helpers, isWithinLimit, canUseFeature)
- ✅ Repository Interface + Eloquent Implementation
- ✅ SubscriptionService: getPlans, startTrial, subscribe, activate, cancel, processExpired, checkLimit, canUseFeature

- ✓ Resources: PlanResource (pricing format IDR), SubscriptionResource
- ✓ SubscribeRequest (Enum validation)
- ✓ SubscriptionController: plans, current, subscribe, cancel
- ✓ Routes: public plans + protected subscription endpoints
- ✓ Middleware: EnsureActiveSubscription (Redis cache 5 mnt), CheckFeatureLimit
- ✓ Artisan Command: `velora:check-subscriptions` (scheduler daily)
- ✓ Seeder: 3 plans (Starter Rp99rb, Growth Rp299rb, Enterprise Rp799rb)
- ✓ Listener: StartTrialSubscription (queued, dipanggil saat UserRegistered)
- ✓ Business Model update: `subscription()`, `hasActiveSubscription()`, `currentPlan()`

Next: Module POS — Products, Categories, Variants, Units, Barcodes